Self-Improving LLMs and Agents for Network and Software Security: A Survey

Anonymous Author(s)

Additional Key Words and Phrases: multi-agent collaboration, large language models, agent orchestration, evaluation benchmarks, agent security

1 INTRODUCTION

Motivation: domain background and the core tension. Software and network security has become one of the most active application domains for large language model (LLM) agents. Systems in this space detect vulnerabilities, repair them at repository scale, plan and execute penetration tests, audit smart contracts, and red-team both models and the agents built on top of them. Across these otherwise disparate systems, a common architectural shift is visible: rather than issuing a single inference pass, the agent acts, observes an outcome, and modifies some part of itself before the next attempt. What gets modified differs widely—prompts and reusable skill libraries, persistent memory, execution workflows, toolchains, and occasionally model weights —and so does the name attached to the mechanism: self-evolving, self-improving, experience-driven self-skill discovery, and self-reflection.

This terminological proliferation has outpaced definition, and the gap matters most in security settings. The same phrase, self-improvement, is used for reflection that lives and dies inside a single episode and for mechanisms that accumulate artifacts across tasks; these two carry very different evidential weight, yet they are reported interchangeably. Second, security tasks make the feedback loop itself unreliable: ground truth is scarce, static analyzers disagree with human judgement at the per-sample level even when aggregate rates look reassuring, and widely used repair proxies such as compile rate can be satisfied by degenerate patches. Third, the loop is an attack surface—experience composition, persistent control of self-evolving agents, backdooring, endogenous misalignment, and deceptive feedback from honeypots all show that a self-modifying security agent can improve in the wrong direction. Without a shared definition, neither the scope nor the transferability of a self-improvement claim can be assessed.

Novelty: what distinguishes this review from prior surveys. Existing surveys approach this territory from three

2 RELATED WORK

Survey literature on language-model security has so far organised itself around two poles. One pole treats LLM agents as assets to be protected: a systematic review of the self-security protection and cybersecurity empowerment of LLM agents separates the agent-as-victim from the agent-as-tool, while a policy-facing account of agent security considerations catalogues prompt injection, tool misuse and data exfiltration as the prevailing threat classes. The other pole treats agents as instruments: a survey of AI-driven automated program repair and code generation [1] and a synthesis of LLM-plus-multi-agent patching in legacy code [2] both narrate the transition from template-based to model-driven repair. A third and more recent strand asks what happens once agents persist and revise their own state, surveyed under the heading of safety in self-evolving agents. Each of these surveys is organised by threat category or by task family; none is organised by the improvement mechanism that connects the two. Our review differs in exactly this respect: rather than cataloguing what agents break or fix, we ask how feedback available in a security task is converted into a persistent change in agent behaviour, and we make that conversion the primary axis of comparison.

A consistent conclusion across these surveys is that capability and safety are not separable design concerns, and that evaluation practice lags behind capability, with fragmented benchmarks, task-specific oracles and gains that rarely transfer [1]. Where improvement is discussed at all, it is described either as

inference-time context adaptation or as a property to be measured rather than a mechanism to be designed. Red-teaming studies of coding agents reinforce the point from the opposite direction: capability gains and new attack surfaces arrive together, and the assumptions under which either is demonstrated usually remain implicit. The existing survey field therefore supplies vocabulary for threats, tasks and architectures, but not for the loop that closes between a security signal and an agent's subsequent behaviour. The review below is organised by the source of that signal and by the object it updates, an organising principle absent from prior surveys.

Empirical benchmarking of secure code generation forms the largest and most mature group. [3] evaluates code copilots along both security and usability axes and finds the two frequently in tension; positions models as end-to-end secure code agents for Python and shows that security defects survive functional correctness; benchmarks prompt-engineering techniques with GPT models and concludes that prompt design reduces but does not eliminate vulnerability rates; argues that correctness-only benchmarks are inadequate and pairs task-adaptive vulnerability modelling with execution-based evaluation; and reconstructs vulnerability-introducing scenarios to benchmark agentic coding by AI agents. Read together, these studies establish that generation-time security is measurable but strongly benchmark-dependent. Our work takes this measurement apparatus as given and asks a different question: whether a generator can be made to improve across iterations using precisely the signals these benchmarks compute.

A second group moves from measurement to mitigation through data or through structural constraint. [4] conducts a dataset-driven study of vulnerability mitigation and shows that curated vulnerable-fix pairs shift model behaviour more reliably than instruction alone; frames LLM-driven secure generation as a self-fixing process in which a model is guided to repair its own output and reports that guidance quality dominates model scale; constraint-based approaches instead restrict the space of admissible programs, as in, which exploits the Scala type system, and, which shows that lightweight models can produce secure PowerShell once the generation task is narrowed. These works share the assumption that a single corrective step, whether learned or enforced, suffices. We read them as supplying the raw material — supervision data, repair prompts, structural checks — from which multi-round improvement loops can be assembled, and we differ by treating the number and ordering of corrective steps as a design variable rather than a constant.

Retrieval and knowledge injection constitute a third route to improvement. [5] combines retrieval-augmented generation with multi-tool feedback and reports reductions in vulnerabilities, logical inconsistencies and compilation failures; injects security knowledge into retrieval-augmented code generation and demonstrates that retrieved context can itself carry unsafe guidance; replaces flat retrieval with comparative reasoning over secure coding practices; and keeps a human in the loop, showing that persistent human feedback combined with static analysers outperforms fully automatic judging. The common finding is that external knowledge helps most when it is filtered by an executable check. Our review treats retrieval as one feedback source among several and is concerned with when retrieved knowledge should update an agent's stored context rather than only its current prompt, a distinction these systems do not draw.

Multi-agent designs that close the loop with program analysers are closest to our own framing. [6] couples a coding agent, a static-analysis agent and a fuzz-testing agent so that analyser findings condition the next generation step; adapts fuzzing to LLM-generated code by deriving semantic guidance from the specification; [7] hybridises static analysis with an LLM repair agent for Python; targets immature vulnerabilities during pre-integration review, where lightweight analysers are the only available signal; and resolves vulnerability issues end-to-end by combining an agent with repository tooling. These systems demonstrate that analyser output is an effective corrective signal, yet each reports a fixed number of rounds and a fixed role assignment, and none analyses whether additional rounds still help. Our work retains their feedback channels but makes the loop itself the object of study, asking when a signal is informative enough to justify another iteration.

Execution- and test-driven repair supplies the clearest evidence that feedback content matters. [8] studies LLM repair with reasoning and patch-validation feedback and shows that validation reduces semantic drift; formalises repair as process-based feedback over intermediate reasoning steps rather than only over final patches; uses test-driven feedback for low-resource code generation and reports that execution results outperform static prompting; applies model-generated patches to real fuzzing findings in OSS-Fuzz and documents where automated fixes fail. This group establishes a monotonic intuition — more informative feedback yields better patches — but verifies it within a single task and a single round budget. Our review generalises the question across security tasks and asks which part of the signal, rather than how much of it, drives improvement.

Domain-structured repair methods constrain the search space with symbolic knowledge. [9] combines neural generation with symbolic validation for vulnerability repair; guides repair with domain-specific

patterns mined from historical fixes; [10] targets TEE partitioning bugs using a DSL-guided, LLM-assisted patching strategy; examines the obstacles to reliable repair and argues that scanner-reported locations are a weak substitute for semantic understanding. The shared claim is that reusable structure — patterns, grammars, type rules — substitutes for repeated trial and error. Our work agrees that structure is a form of accumulated experience, but locates it inside the agent's memory rather than in an external artefact, which makes it updatable and directly comparable with purely empirical feedback.

Vulnerability detection has itself been reshaped by retrieval and multi-agent prompting, and the detectors produced there double as feedback providers for repair. [11] compares retrieval-augmented, fine-tuned and dual-agent detectors and reports that retrieval augmentation attains the highest detection quality, a result we read as evidence that detector fidelity bounds the value of downstream feedback; applies LLMs to vulnerability detection where classical tooling dominates; [12] grounds analysis in provenance so that retrieved evidence is attributable; evolves prompts across models within a retrieval-augmented multi-agent detector; [13] runs parallel LLM and retrieval components for smart contracts; adaptively mixes experts; and [14] manages smart-contract vulnerabilities within a single framework. The collective lesson is that detection quality is not the end of the story but its input. Our review connects this literature to repair and attack work by treating the detector as a component whose output statistics, including their stability, govern how an agent should react to it.

Autonomous penetration testing is the offensive counterpart to repair. [15] incorporates LLM agents into reconnaissance, planning and exploitation; coordinates specialised agents across multi-host campaigns; [16] scales multi-agent testing to complex environments; pursues a fully automated initial-access workflow and emphasises reliability to first shell; [17] adapts domain-specific models to the offensive task; and targets dynamic web applications produced by AI-assisted development. These systems are typically evaluated by success rate or attack success rate on fixed targets, and their use of past experience remains implicit in notes or memory. Our work differs by asking how offensive experience should be represented and reused, and by treating transfer to unseen targets as the quantity that separates genuine improvement from target-specific memorisation.

A subtler line replaces brute-force exploration with structured knowledge and reflection. [18] makes self-reflection central, letting an agent revise its plan against accumulated knowledge; constrains reasoning with structured attack trees; [19] grounds execution in a knowledge base so that steps are retrievable and auditable; formulates adaptive distributed testing game-theoretically with neuro-symbolic components; and extends red-teaming to quantum-resistant cryptographic standards, showing that domain knowledge dominates generic planning. These works show that structure improves sample efficiency, yet they rarely quantify how much of the gain comes from structure and how much from additional attempts. Our review performs exactly this decomposition, and we treat structured knowledge as one candidate memory substrate to be compared against unstructured experience rather than assumed superior.

Reinforcement learning offers the most explicit formulation of improvement as policy change. [20] rewards behavioural diversity and curiosity in multi-objective deep RL for automated testing; trains self-correcting agents to trade security against performance in infrastructure control; [21] combines RL with semantic rewards for vulnerability repair; applies coverage-guided RL to mutation in JavaScript engine fuzzing; and trains models to self-evolve, making the improvement procedure itself a learned skill. The attraction is a principled objective, but the cost is that rewards are task-specific and seldom encode security semantics directly. Our work stays with non-parametric adaptation through memory and context precisely because security rewards are sparse and expensive, and we use these RL results as an upper bound on what explicit optimisation can achieve.

LLM-guided fuzzing contributes both a class of improvement signals and a class of evaluation targets. [22] guides smart-contract fuzzing with generated inputs and properties; synthesises over the space of fuzzers rather than over inputs alone; [23] shows that feedback channels inside a deep-learning framework can be exploited for bug finding; mutates inputs with semantic feedback in a hybrid setting; [24] augments fuzzing harnesses for unsafe Rust through LLM generation; and aggregates multiple feedback sources for smart contracts. Coverage and crash signals in these systems are dense and cheap, in contrast with the sparse signals available to repair agents. Our review uses this contrast to argue that feedback density, more than feedback type, predicts whether iterative self-improvement is worth attempting.

A rapidly growing body of work turns the same fuzzing and feedback machinery against agents themselves. [25] fuzzes agentic browsers for prompt injection in real time; targets workflow-level multi-tool vulnerabilities with greybox fuzzing; [26] fuzz-tests text-to-image models to elicit jailbreaks; attributes tool invocations causally to defeat indirect prompt injection; and red-teams coding agents from a tool-invocation perspective and documents which assumptions make attacks succeed. These studies reveal that an agent's own adaptation is an attack surface as well as a defence. Our review takes this duality seriously:

we treat adversarial feedback as one of the signals an improving agent may incorporate, and we require any claimed improvement to be evaluated under an adaptive adversary rather than a fixed attack suite.

Defensive countermeasures have been proposed at three levels. At the interaction level, [27] proposes a lifelong guardrail that adapts its safety detection to an agent's evolving behaviour; at the system level, coordinates multiple agents into a self-reflective defence; and at the evaluation level, replaces static red-teaming with an evolutionary evaluation in which attacks and defences co-evolve. Persistence introduces a distinct hazard: [28] shows that long-term memory allows an attacker to retain control across sessions, so that self-evolution becomes a liability rather than an asset. Policy-oriented analysis [29] and systematic review [30] note the same asymmetry but stop short of mechanism-level guidance. Our work shares the co-evolutionary premise of [31] and extends it by asking which stored artefacts must be validated before they are permitted to persist.

The generic self-evolving agent literature supplies mechanisms that security work has yet to adopt. [32] frames improvement as context engineering, curating what an agent stores and replays rather than changing its weights; turns self-evolution into a trained capability; [33] discovers reusable skills from experience in clinical imaging; builds a self-evolving security framework for 6G networks; and [34] and apply evolving cognitive agents to phishing and DDoS defence respectively. [35] surveys the safety questions that arise once state persists, including the compression and abstraction of stored experience. Notably, only a minority of these systems are evaluated on tasks with ground-truth security oracles. Our review's central observation is that this literature is mechanism-rich and security-poor: we import its mechanisms and subject them to the adversarial, oracle-constrained conditions that security tasks impose.

Domain-specific verification settings show what a trustworthy oracle looks like when one exists. [36] generates threat models and test plans for hardware security verification; applies agentic verification to system-on-chip designs; [37] plans and executes smart-contract audits with a dynamic plan-execute structure; and retrieves properties for formal verification so that generated specifications are machine-checkable. In these domains success is decided by a checker rather than a judge, which makes iterative improvement measurable in a way that scanner- or judge-based evaluation is not. Our review uses them as existence proofs: they show that when verification is cheap and sound, feedback-driven improvement converges, and they delimit the conditions under which the same claim may be made for fuzzing- and scanner-backed security tasks.

Taken together, existing work covers threats, tasks, architectures and, increasingly, mechanisms for self-improvement, yet the coverage is uneven in three specific ways. First, mechanism-level taxonomies are absent above the level of individual systems: methods proliferate but are seldom compared under a shared vocabulary [38, 35]. Second, feedback sources are studied in isolation, so detection scores [11], analyser and fuzz signals, and attack outcomes [15, 39] are never evaluated within one framework. Third, the assumptions that make an improvement claim true, such as black-box interaction, query and round budgets, or accessibility of model weights, are stated per paper and never accumulated [40, 31]. Benchmark fragmentation compounds all three. This review addresses these gaps by organising prior work along feedback source, updated object and update timing, by compiling the assumption sets that condition each claim, and by marking where the evidence is too thin to support the transferability assertions currently made.

The following works act as hubs in the literature reviewed above, each anchoring several of the strands discussed.

- [5] couples retrieval-augmented generation with multi-tool feedback, an early demonstration that heterogeneous analyser signals can be combined into a single corrective step for secure code generation.
- [4] provides the dataset-driven account of vulnerability mitigation that supplies much of the supervision data reused by later secure-generation work.
- [41] studies guided self-repair of model output and is the closest antecedent to treating correction as an iterative rather than one-shot process.
- [15] establishes the agentic penetration-testing template of reconnaissance, planning and exploitation that later offensive frameworks extend.
- [39] generalises agentic testing to multi-agent collaboration, making role structure an explicit design variable.
- [6] closes the generation loop with static analysis and fuzz testing, the clearest instance of analyser feedback conditioning subsequent generation.

- [42] carries agentic assistance through to repository-level vulnerability issue resolution, linking detection, repair and validation in one workflow.
- [32] reframes self-improvement as context engineering, supplying the mechanism vocabulary that security-oriented agent work has yet to absorb.
- [40] red-teams coding agents through tool invocation and makes explicit the assumptions under which agentic capability becomes risk.
- [11] compares retrieval-augmented, fine-tuned and dual-agent detectors, quantifying how detector fidelity bounds the usefulness of downstream feedback.

3 SURVEY METHODOLOGY

Of the 141 screened publications, 54 are cited in the synthesis; the remaining 87 informed screening and background and are not individually cited.

Data Mapping. We assembled a corpus of 141 papers through multi-source retrieval that prioritized arXiv and fell back to open academic indexes when arXiv coverage was insufficient. Each record was anchored to a DOI or other persistent identifier, and duplicates were removed by identifier matching before screening; this left 141 unique papers spanning the five research questions and fifteen sub-questions that structure the review. The mapping stage therefore fixed both the provenance of each paper and its assigned question scope, so that later analysis could not silently expand or contract the evidence base.

After clarifying the assumptions under which agent capabilities translate into risks and examining how detector fidelity constrains downstream feedback utility, the relevant research evidence needs to be organized systematically to support cross-question comparison and synthesis. To this end, data mapping of literature sources, identifier anchoring, and deduplication screening becomes the starting point for moving from mechanistic judgment toward empirical induction.

Data Refinement. We fixed the evidence set before analysis began: no paper was added or removed after the 141-paper corpus had been screened against the five research questions and fifteen sub-questions. Each paper was coded into six evidence fields—research problem, method, datasets and benchmarks, metrics, limitations, and assumptions or constraints—and we recorded cross-paper relations such as shared benchmarks, method lineage, and directly contrasting results. This coding produced a structured evidence matrix in which every cell is traceable to a specific paper and field, allowing comparisons to be made consistently across questions rather than paper by paper.

Data Evaluation. For each of the fifteen sub-questions, we analyzed the fixed evidence matrix through three operations: comparison across papers that address the same sub-question, evolution of methods and findings over time, and explicit trade-off analysis among accuracy, cost, generality, and assumptions. These operations were applied per sub-question, not aggregated prematurely, so that disagreements among the 141 papers remain visible. Where the evidence supported a synthesis, we decomposed the conclusion into checkable claims and annotated each claim with the papers that support it; where the evidence did not support a synthesis, we recorded the gap rather than inferring a result.

Data Evaluation (continued). The resulting claims are therefore auditable at the level of individual papers: each claim names its supporting studies, and each sub-question report states which of the six evidence fields supplied the basis for comparison, evolution, or trade-off. This three-layer evaluation—evidence matrix construction, per-sub-question analysis, and claim-level verification—keeps the review's conclusions bounded by the 141-paper corpus and by the fifteen sub-questions fixed in advance.

4 SELF-IMPROVEMENT MECHANISMS FOR SAFETY TASKS IN LARGE MODELS ARE DECOMPOSABLE

4.1 How do existing works each define "self-improvement"? What are the sources of their feedback signals (vulnerability de…

4.2 The Inheritance Topology of Self-Improvement Methods: Five Reusable Mechanism Families and the Missing Horizontal Comparison

The preceding subsection classified self-improvement methods by the provenance of their feedback signals; a complementary structural question now arises: how do the 385 safety-oriented methods surveyed here relate to one another through inheritance and composition, do those relations condense into a small set of reusable mechanism families, and why does the literature record 277 explicit inheritance relations against only 41 explicit comparisons? The short answer developed below is that inheritance is hub-dominated and

vertical rather than lateral, that it collapses into five recurrent mechanism families whose legitimacy is corroborated by existing taxonomies, and that explicit comparison is largely absent because each family is evaluated on a benchmark surface that the others do not touch.

Inheritance in this literature is markedly star-shaped rather than mesh-like. A small number of methods absorb the overwhelming share of descendants: PentestAgent [15] is extended by fifteen works, Agentic Context Engineering [32] by eleven, CyberEvolver [43] by eight, and LLMpatronous by six, while a second tier is formed by VRpilot [8] with thirteen, the retrieval-augmented secure-code workflow with twenty-two, and MemRepair [44] with eight. The distribution is therefore heavy-tailed: a handful of ancestors account for a disproportionate fraction of all lineage statements, and the remaining methods either attach to these hubs or terminate a branch. Such a profile is characteristic of a field that consolidates around a few canonical formulations before it begins to compare them.

This topology carries an interpretive consequence. Because descendants attach to a single ancestor rather than to several, inheritance mainly records the transplantation of one mechanism into a new task domain, not the synthesis of several mechanisms into a stronger whole. The hubs function as canonical carriers of a mechanism — automated penetration testing in the case of [15], evolving contexts in the case of [32], structured on-the-fly evolution in the case of — and their descendants vary the application setting while preserving the core loop. The lineage is thus deep and narrow, which explains why the field accumulates many variants of the same idea but few statements about which idea is preferable.

The first and largest family performs external knowledge or retrieval injection. Here the agent's improvement is sourced from evidence outside its parameters: LLMpatronous [45] supplies vulnerability knowledge to the detector, the retrieval-augmented workflow of grounds secure code generation in retrieved exemplars and tool feedback, and RefPentester conditions reflective penetration testing on curated knowledge. A controlled empirical evaluation of RAG, supervised fine-tuning, and dual-agent designs for vulnerability detection [11] is instructive precisely because it holds the task fixed while varying the knowledge channel, and it indicates that retrieval helps most where the model's parametric knowledge of the vulnerability class is thin. The family's internal diversity is thus about what is retrieved rather than whether retrieval occurs.

The second family closes a verification loop, and its members differ chiefly in verification depth. SecureFixAgent [7] and AutoSafeCoder couple repair to static analysis and fuzz testing, so the improvement signal is executable and adversarial; EvoRepair converts repair experience into reusable guidance across attempts; and small, targeted test suites have been shown to serve as economical improvement signals for self-evolving agents. The contrast is consequential: test-suite signals are cheap and locally faithful but shallow, whereas fuzz-and-static signals are expensive yet expose failure modes that unit tests miss. This trade-off between signal cost and signal depth recurs in every family that depends on external verification, and it explains much of the variance in reported gains.

The third family maintains experience memory or persistent context, and its members diverge on memory granularity. MemRepair [44] organizes repository-level repair knowledge hierarchically, MEMBRANE stores contrastive safety successes and failures as a defence memory, and the skill-memory agent of [46] persists procedural knowledge across episodes; Agentic Context Engineering [32] treats the context itself as the evolving artifact, while SkillProx updates skills through proximal textual-gradient steps. Finer granularity yields more transferable units but demands stronger retrieval discipline; coarser granularity is easier to maintain but risks entangling irrelevant history. The mechanism underlying all of them is the same — convert past trajectories into a persistent substrate that conditions future behaviour — yet no shared protocol governs what should be written to that substrate.

The fourth family evolves the scaffold itself. CyberEvolver [43] mutates the agent's structured workflow on the fly, transferable self-evolving playbooks evolve reusable audit procedures that carry across targets, and a systematic study of feedback dynamics over multiple rounds shows that scaffold-level gains are non-monotonic and can decay as rounds accumulate. SmartAuditFlow [37] instead fixes a plan-execute schedule whose adaptivity lies in re-planning rather than in structural mutation. The families therefore differ in what is mutated — control flow, textual procedure, or schedule — and the evidence in suggests that structural mutation without a stabilizing criterion is the most fragile of the three.

The fifth family orchestrates multiple agent roles. PentestAgent [15] assigns reconnaissance, exploitation, and reporting to distinct roles, VulnBot coordinates a multi-agent penetration-testing team, and RefPentester [18] adds a reflective reviewer that critiques prior role outputs; AutoSafeCoder [6] similarly separates generation, static analysis, and fuzz testing into cooperating agents. The design axis here is whether role assignment is static or itself revisable: static decomposition buys interpretability and parallelisability, whereas revisable decomposition buys adaptation at the cost of coordination overhead. Compared with scaffold evolution, role orchestration changes who performs a step rather than which steps

exist, a distinction that the literature rarely makes explicit.

These five groupings are not an artefact of the present reading; they align with independently proposed classifications. A framework for self-evolving coding agents organizes the field by the carrier of persistent change — skills, workflow, memory, framework, and model — while a unified framework for self-evolving and self-healing agents proposes four axes covering the object of change, the timescale of adaptation, the recovery boundary, and the guarantee mechanism. A survey of safety in self-evolving agents reaches compatible conclusions about where change can be lodged [35]. The five families identified here appear to correspond to mechanism-level projections of these carrier-level axes, which lends the decomposition external support rather than merely internal consistency.

Against this clear familial structure, explicit comparison is strikingly sparse: only 41 comparison relations accompany the 277 inheritance relations, and almost

4.3 Temporal Stratification of Self-Improvement Evidence

This subsection asks how empirical support is distributed across the four temporal scales on which self-improvement is claimed to operate—single-round feedback, multi-round iteration, cross-task experience accumulation, and cross-version continual evolution—and how fixed iteration budgets truncate what can be observed at the long end of that spectrum. The distribution is strongly front-loaded: support is dense and quantitative at the single-round scale, bounded by design at the multi-round scale, moderate yet gain-limited at the cross-task scale, and thin at the cross-version scale, where one simulation-based longitudinal study and one temporal-split study carry most of the weight. Because each tier is operationalised through a fixed budget rather than treated as an experimental variable, the observation window is set by configuration choices rather than by the dynamics under study. The paragraphs below trace this gradient tier by tier, identify the censoring effect that bounded budgets impose on reported outcomes, and mark precisely where the evidence thins out.

The single-round tier is by far the most saturated, and its saturation is explained by the density of the feedback signal rather than by any property of the mechanisms themselves. VRpilot couples reasoning traces with patch-validation feedback and reports 14% more correct patches on C and 7.6% more on Java relative to its comparison condition, a gain obtained without any iterative re-entry. SecureFixAgent goes further in methodological terms by elevating the average number of convergence iterations to a first-class evaluation metric, thereby converting a binary success indicator into a distribution over effort. The underlying mechanism is straightforward: patch validation yields a cheap, per-attempt, externally verifiable signal, so a single feedback round already carries enough information to close the loop. Where such a verifier exists, self-improvement is largely a one-shot correction problem, and the abundance of evidence at this scale reflects the low cost of running the experiment rather than the strength of the long-horizon claim.

The multi-round tier, in contrast, is locked shut by hard-coded budgets. AutoSafeCoder caps its static-analysis feedback loop at four iterations and acknowledges in its own limitations that four rounds may not resolve all vulnerabilities [6]. A revealing comparison is SecureFixAgent, which also operates under a bounded loop but responds to residual failures by substituting the feedback source—a Bandit rescan—rather than by extending the horizon. The two systems therefore differ in re-entry policy

5 EFFECTIVENESS STRATIFICATION OF FOUR TYPES OF SAFETY FEEDBACK SIGNALS

Once self-improvement for safety tasks is decomposed into its constituent mechanisms, the decisive question becomes which feedback signals actually sustain measurable gains—and the evidence shows that effectiveness is stratified rather than uniform across signal families. The governing tension is that gain tracks signal fidelity more than signal type: vulnerability-detection feedback scales with detector precision and collapses under alarm noise when explicit safety specifications are absent, runtime feedback (crash reproduction, test execution, sanitizer signals) is repeatedly validated as a direct driver while coverage evidence remains sparse and thin, and attack–defense experience signals report substantial gains under evaluation paradigms, metric semantics, and environmental assumptions that are not strictly commensurable with those of the vulnerability and repair sides. The chapter accordingly traces this stratification from detection feedback, through runtime and coverage signals, to experience-based defense, isolating in each case the conditions under which a signal reliably closes the loop. These conditions also delimit where self-improvement ceases to transfer, which motivates the task- and scale-dependent boundaries examined next.

5.1 Signal-to-Noise Conversion in Detector-Driven Feedback: Precision Illusions, Recall Collapse, and Closed-Loop Instability

This section examines how the precision, recall, and F1 of vulnerability detection results—static analyzer alerts, CWE determinations, and detector confidence—convert into self-improvement gains when they are recycled as feedback, and how alert noise destabilizes the closed loop under the absence of explicit security specifications. Does a detector with higher aggregate precision necessarily yield a more executable self-improvement signal? The evidence assembled here indicates that the conversion is governed less by headline detection scores than by per-sample reliability: gains scale with the signal-to-noise ratio of individual judgments, and once that ratio degrades, the loop does not merely improve more slowly but becomes actively unstable, recurring on the same weakness classes or drifting toward attacker-specified repairs. We develop this argument in three movements: the stratification of detection quality and its translation into feedback value, the mechanisms through which noise erodes loop stability, and the mitigation strategies that recover stability by replacing raw rule alerts with fused or semantic criteria.

The first strand of evidence establishes a monotone relationship between detector quality and feedback utility. In a controlled comparison of detection strategies, retrieval-augmented detection reached an average F1 of 0.85, supervised fine-tuning 0.80, a dual-agent configuration 0.77, and an unassisted model only 0.67 [11]. The roughly eighteen-point F1 gap between retrieval-augmented and baseline detection is not a cosmetic benchmark difference; it corresponds to whether a flagged location and its associated weakness type are specific enough to be acted upon by a downstream repair step. Retrieval grounding supplies the contextual precedent that makes an alert interpretable, whereas an unassisted detector produces judgments whose justification cannot be transferred into a repair instruction. This gradient therefore defines the upper bound of what detector-derived feedback can contribute: high-F1 detectors carry actionable signal, and weaker detectors carry claims that a repair component must either verify independently or discard.

Aggregate precision, however, is a treacherous proxy for that per-sample reliability. Evaluating Semgrep and CodeQL against 1,080 human-annotated samples produced by GPT-4o, the two analyzers agreed with human judgment on only 65% and 61% of individual cases respectively, even though CodeQL's aggregate classification of samples as secure reached 80% against a human rate of 61%. The discrepancy reveals an aggregation illusion: a tool can appear conservative and accurate at the corpus level while its case-level verdicts diverge from ground truth at rates approaching two in five. Because a feedback loop consumes case-level verdicts one at a time, the aggregate figure never enters the loop—only the divergent per-sample judgments do. Consequently, reported analyzer precision overstates feedback quality precisely where it matters, and the ~61–65% per-sample agreement band marks the empirical threshold below which detector output should be treated as a noisy prior rather than an authoritative signal.

Recall failures compound the problem and take two distinct forms. On 46 known weakness locations in the Autoware codebase, CodeQL and Semgrep jointly achieved zero hits, while taxonomy-guided LLM analysis recovered up to 76% of the same locations. This is silence rather than noise: the feedback channel reports nothing, so the loop terminates in the false belief that no defect remains. The opposite pathology appears with cppcheck, which emitted 1,301 alerts in a single run, a volume that no iterative repair process can consume without extensive triage. Silence and flooding are thus two ends of the same recall-calibration failure, and they demand different remedies: the former requires a detector with semantic reach beyond pattern matching, the latter requires filtering before feedback is issued.

A further constraint arises from the correlation between generator and detector when both are the same model. End-to-end evaluation of LLMs as secure code agents reports self-audit accuracy of 43.6% for GPT-3.5 and 74.6% for GPT-4, with repair success on self-produced code capped near 60% and an average of 76.2% of code remaining insecure on SecurityEval. The mechanism is shared blind spots: when the auditing model and the generating model fail on the same constructs, detector confidence ceases to be independent evidence and instead reproduces the generator's own error distribution. This explains why self-feedback saturates well below the reliability implied by standalone detection benchmarks, and it suggests that any single-model loop has an intrinsic ceiling regardless of iteration budget.

Closed-loop deployments confirm that these signal deficiencies manifest as instability rather than slow convergence. AutoSafeCoder drives repair with Bandit static-analysis feedback but constrains the loop to four iterations, and within that budget CWE-94 recurred 25 times across 44 vulnerable samples [6]. Recurrence of the same weakness class after repeated feedback is the signature of a loop that is not accumulating corrective information: the analyzer's verdicts neither confirm the fix nor localize the residual defect, so the repair component oscillates within the same failure basin. An important caveat is that the iteration cap and the noise are confounded—recurrence may be partly attributable to the hard budget rather than to feedback quality alone—yet the co-occurrence of a tight cap and high recurrence is itself

consistent with practitioners compensating for unreliable signals by limiting exposure.

The absence of explicit security specifications converts this instability into a security hazard rather than a quality nuisance. When defect reports are adversarially constructed and no formal specification constrains what a correct patch must satisfy, more than 90% of generated patches conform to attacker intent, and the strongest filter examined blocked only 47%. The result reframes the role of specifications: without an explicit statement of the intended security property, the feedback signal is the only arbiter of correctness, and a noisy or manipulable arbiter can be steered. Noise in this condition is therefore not zero-mean; it is partially adversarial, which is why stability analyses that treat alert noise as random perturbation understate the risk.

Two mitigation families recur across the evidence, and they differ in where they place trust. The first fuses heterogeneous signals before feedback is issued: combining GCC, KLEE, and CodeQL while limiting repair to three rounds improves outcomes relative to relying on any single analyzer's verdict [5]. Fusion helps because the tools fail on different constructs, so agreement raises effective precision and disagreement flags cases for verification. The second family replaces raw rule alerts with semantic judgments. SAFuzz employs an LLM-based semantic vulnerability predictor, evaluated by ROC-AUC, precision, and bug-detection recall, to allocate fuzzing effort adaptively and filter non-vulnerable targets before they consume budget [47]; AgenticSCR pairs a detection sub-agent with a verification sub-agent and a security semantic memory to process alerts described as noisy and context-poor, scoring outcomes by localization, type correctness, and relevance. Both approaches accept that static alerts are unreliable and insert an interpretive layer rather than trusting the rule.

Comparing the two families clarifies their respective operating conditions. Fusion is cheaper and requires no additional model capacity, but it inherits the recall ceiling of its constituent analyzers—if all fused tools miss a weakness class, as CodeQL and Semgrep did on all 46 Autoware locations [48], agreement cannot recover it. Semantic verification extends recall beyond pattern matching and can adjudicate context, but it introduces dependence on the verifier's own competence, which the self-audit results bound at 74.6% even for a strong model [49]. The evidence therefore suggests a division of labor rather than a competition: fusion for precision control at low cost, semantic verification for recall extension where contextual reasoning is required, and explicit specifications where adversarial pressure is plausible.

Taken together, the failure modes form a compact taxonomy that maps each signal deficiency to a distinct loop pathology. Recall collapse produces silent termination, where the loop stops because no alert is raised [48]; precision inflation at the aggregate level produces misplaced confidence, where divergent case-level verdicts are treated as authoritative [50]; alert flooding produces triage saturation, where feedback volume exceeds processing capacity [48]; and specification absence produces objective corruption, where the loop optimizes toward an attacker-supplied target [51]. These pathologies couple: flooding raises the cost of verification, which in turn makes aggregate-precision heuristics more attractive, which in turn increases susceptibility to corrupted objectives. Recognizing the coupling matters because single-mechanism fixes address one pathology while leaving its downstream amplifiers intact.

The boundaries of this evidence deserve explicit statement. No study in the assembled literature reports a sensitivity ablation that varies the proportion of noisy alerts and measures convergence behavior, so the shape of the noise–stability curve—and in particular whether a sharp threshold exists near the observed 61–65% per-sample agreement band—remains unmeasured. Metric definitions are also heterogeneous: the F1 of 0.85 describes a retrieval-augmented detector, whereas the 61–65% figures describe per-sample agreement of conventional static analyzers, and the two are not directly commensurable [11, 50]. Most loop studies impose fixed iteration budgets, and most evaluations are confined to Python or to a single codebase, leaving cross-language generality untested [6, 49]. These limitations qualify the magnitude of the claims above without altering their direction.

In summary, the self-improvement gain obtainable from vulnerability detection feedback is bounded by per-sample signal quality rather than by aggregate detector scores, and under the absence of explicit security specifications, alert noise acts as a primary destabilizer that manifests as non-convergence, weakness recurrence, and susceptibility to adversarial steering. The effective remedies—multi-tool fusion and semantic verification layers—all operate by raising the effective signal-to-noise ratio before feedback enters the loop. This conclusion motivates the next section, which turns from detector-derived signals to the specification-bearing and human-anchored feedback channels, where the arbiter of correctness is supplied externally rather than inferred from alert statistics.

5.2 Program runtime feedback (crash reproduction, test execution results, dynamic analysis/sanitizer signals) and test…

5.3 Attack–defense experience signals (Attack success rate / ASR, penetration trajectories, experience…

6 GENERALIZATION OF SELF-IMPROVEMENT IS CONSTRAINED BY TASK AND SCALE BOUNDARIES

Efficacy stratification alone is not sufficient to establish that the observed gains continue to hold beyond their measurement conditions. The survey corpus shows that the generalization of self-improvement is not a property that can be extrapolated smoothly; rather, it is a property jointly bounded by three constraints—language, granularity, and task form. These three constraints compound rather than cancel one another, and authors also prefer to use explicit scope-of-applicability assumptions to delimit what their conclusions can claim. The distributional traces left by the three constraints, the explicitly stated narrowing assumptions, and out-of-distribution evidence form a gradient of decreasing evidential strength: the first two remain checkable, whereas out-of-distribution evidence is sparse and largely supplementary in nature, so claims of cross-boundary generalization are therefore the weakest.

6.1 How severe is the distribution skew of the evaluation benchmarks (HumanEval 6 / SecurityEval…

6.2 Which generalization-narrowing assumptions do existing works explicitly declare (Isolated-function eval…

6.3 Does evidence of self-improvement exist for cross-language transfer, generalization across CWE families, and real-repository-scale deployment? In hea…

(Note: The original text is truncated at "cross-bound…" and is here closed as "cross-boundary generalization" based on context; the placeholder

7 GENERALIZATION OF SELF-IMPROVEMENT IS CONSTRAINED BY TASK AND SCALE BOUNDARIES

–

7.1 Is there evidence of self-improvement in cross-language transfer, cross-CWE-family generalization, and real repository-scale deployment? In hea…

is retained in place, and no citations have been added or removed.)

8 SAFETY-SPECIFIC JUDGES DETERMINE TRUSTWORTHINESS MEASUREMENT

Measured trustworthiness in safety-critical self-improvement hinges on what the evaluation actually executes: whereas general metrics such as accuracy, recall, and pass rates dominate reported associations, the far smaller set of safety-specific judges—those that verify exploitability or otherwise ground verdicts in executable evidence—carries the decisive weight in determining whether a claimed improvement is trustworthy. This asymmetry is compounded by the near-absence of functionality-preservation constraints as a measurable construct, which leaves conclusions about patch-style self-improvement vulnerable to validity threats, and by an argument topology in which inheritance and weak comparison far outnumber genuine support, so that claims of mechanism efficacy rest on thin evidential scaffolding and remain exposed to self-authored metrics, benchmark and evaluation contamination, and cross-paper incomparability. The chapter therefore proceeds from the composition of safety-specific versus general measurement, through the gap in functionality-preservation metrics and its consequences for validity, to the fragility of the resulting evidentiary structure—whose dependence on fixed threat assumptions and static evaluation settings ultimately motivates the shift toward transferability under evolving threat models and co-evolution.

8.1 Proxy Substitution and the Measurement Bias of Absent Security Oracles

This section asks how the 256 metrics inventoried in the preceding chapter divide between general-purpose measures and security-specific oracles, and what systematic bias follows when the latter are replaced by proxy indicators. The short answer is that the metric vocabulary of this literature is overwhelmingly

general-purpose: accuracy-, recall- and pass-rate-style measures recur across dozens of studies, whereas oracle-grade evidence about whether a vulnerability has actually been removed appears only in isolated studies and is almost never reused as a shared evaluation instrument. That asymmetry is not a neutral descriptive fact about scholarly taste. It shapes what the field can claim to know, because a general-purpose measure imported from classification or code-generation benchmarking carries an implicit theory of what security is, and that theory is rarely examined. The analysis below first quantifies the composition of the metric inventory, then traces three substitution chains through which security-specific oracles are displaced, and finally identifies the coupled biases that these substitutions introduce into reported trustworthiness.

The distribution of metric usage is markedly skewed toward general-purpose instruments. Security-specific oracle measures — per-sample label agreement between a judge and human annotation, the yield of samples on which independent scanners agree, and explicit verification success — each trace to a single study, and none has been adopted by three or more studies as a primary criterion for judging repair output [50]. General-purpose measures, by contrast, recur systematically: Accuracy with ten instances, Recall with eight, Pass@1 with six, Precision with six and F1 with five, alongside proxy indicators such as compile rate and plausible-patch rate that are counted informally rather than as first-class metrics. The asymmetry is therefore not merely numerical but structural: reuse begets reuse, since a measure that already appears in a dozen papers requires no justification, while a bespoke oracle must defend its construct validity from scratch in every study that proposes it. The consequence is a measurement economy in which commensurability is purchased at the cost of construct fit, and in which the metrics most likely to detect a security failure are precisely those least likely to be applied.

The root cause of this displacement is ontological rather than technical: the target property is a negative one. A patch is secure when no exploitable path remains, and absence of exploitability admits no direct executable test in the general case, whereas compilation, test-suite passage and similarity to a reference fix are all positively observable. Substitution is thus a rational response to an observability gap, and the question is not why proxies are used but which proxies preserve enough of the target construct to support inference. Compile rate preserves almost none of it. Five controlled experiments reported in [52] show that it is insensitive to genuine improvement, that it can vary by a factor of 2.7 under a single compiler flag, that it produces rankings opposite to those obtained with CodeBLEU, and that it is trivially gamed by deleting code, with degenerate patches still scoring between 0.08 and 0.48. A metric with flag-dependent variance of that magnitude cannot discriminate between systems whose true difference is smaller than the flag effect, and the gaming result shows that the proxy rewards the very behaviour a security judge should penalise.

A second substitution chain treats static analysers as stand-in security oracles, and its bias is of a different character. Using 1,080 human-annotated ground-truth samples, [50] reports per-sample label agreement of only 61% for CodeQL and 65% for Semgrep, even though the aggregate security rates look reassuringly close: 61% for human annotation, 60% for Semgrep and 80% for CodeQL. The gap between aggregate agreement and per-sample agreement is the signature of an ecological fallacy — the analysers agree on the base rate while disagreeing about which samples belong to it. Studies that compare secure-code generation techniques therefore inherit a labelling process whose error is invisible at the level of the reported mean, a concern that applies directly to prompt-engineering comparisons of the kind conducted by. The practical implication is that two systems with statistically indistinguishable aggregate security rates may be correct on largely disjoint sample sets, and no amount of additional sampling from the same scanner will reveal this, because the bias is in the instrument rather than in the sample.

The third and most severe variant of scanner-as-oracle is definitional rather than stochastic.

8.2 Metric Invention, Contaminated Benchmarks, and Incommensurable Measures: Validity Risks under Thin Evidence

How robust is the claim that a given self-improvement or repair mechanism "works" when the surrounding literature offers only one explicitly contrary finding against six supporting ones, and does that thin dialectic conceal metric invention, benchmark contamination, and cross-paper incomparability? Our reading of the available evidence is that robustness is low on all three counts, and that the shape of the supporting argument is itself diagnostic: agreement clusters on negative propositions about tooling rather than on positive propositions about mechanisms, so the field's confidence rests more on shared scepticism about proxies than on adversarial testing of the mechanisms themselves [50][52]. This section substantiates that judgement by examining where measures come from, what they are computed on, and whether they mean the same thing across studies.

The distribution of agreement is itself a measurement of how much scrutiny a claim has survived.

Across the surveyed work, most inter-paper relations take the form of extension or loose comparison, whereas genuine confirmation concentrates on the negative proposition that static analyzers are unreliable as ground truth; this proposition is corroborated from independent angles, including end-to-end secure code generation [49] and taxonomy-driven weakness discovery [48], and reinforced by direct empirical failure analysis of repair metrics [52]. By contrast, the positive proposition—that a specific self-evolution or repair mechanism yields real improvement—attracts almost no refutation. Absence of counter-evidence should not be read as robustness: it more plausibly indicates that the community has not yet attempted systematic falsification, which makes the positive claims resemble consensus inertia rather than tested knowledge.

Metric invention is the most directly observable risk. One conceptual framework introduces at least eight measures—adaptation gain, retention, transfer, false recovery, degradation area, detection delay, time to recovery, and safety violations—while explicitly reporting no new empirical benchmark results, with Defects4J appearing only as a cited substrate. Each of these measures is invoked by that single paper and by no other study in the surveyed corpus, so none has been subjected to independent reuse, calibration, or correlation with human judgement. The mechanism is familiar from measurement theory: a measure

9 TRANSFERABILITY IS CONSTRAINED BY THREAT MODELS AND CO-EVOLUTION

Trustworthiness measurement, as the preceding chapter establishes, rests on safety-specific judges whose judgments are defined relative to a particular threat model; transferability therefore cannot be treated as an intrinsic property of a model but only as a function of how explicitly that threat model is specified and how both attackers and defenders evolve against it. This chapter develops that tension: constraints that papers assert in prose seldom survive as structured relations, leaving the assumptions governing a judge's validity largely unmodelled and hence silently non-portable across deployment contexts. Where co-evolution is claimed, the evidence is overwhelmingly one-sided, with an adaptive attacker or defender measured against a fixed counterpart rather than two learning agents closing the loop, and the limitations that accumulate along the experience-update-reuse cycle compound into structural failures rather than isolated ones. Transferability is thus argued to be conditional on explicit threat modelling and genuinely bidirectional co-evolution, and the sections that follow examine each condition in turn.

9.1 100 AssumptionConstraint instances (approximately 60 items with no relationship edges) …

9.2 The Co-Evolution Gap: Adaptive Attackers, Stationary Defenders

The sparse relational fabric surrounding assumption and constraint statements, noted above, is symptomatic of a wider property of this literature: declarative commitments are recorded far more often than the adversarial dynamics that would test them. To what extent do current systems actually realise bidirectional co-evolution when both sides are instantiated as LLM agents or agentic pipelines, and how large is the distance between the concept and the evidence? Three observations organise the answer. Self-evolving attackers are empirically mature and quantified; self-evolving defenders exist

9.3 Limitation Families and the Benchmark-to-Production Failure Cascade

The question this section addresses is which families of limitations recur across self-improving agent systems, and how far those families explain why methods that succeed on benchmarks fail once deployed. Coding the limitations reported across the surveyed corpus yields five recurring families—cost, environment fidelity, data availability, evaluation validity, and safety risk—that are neither equally documented nor independent of one another. The central claim developed here is that the benchmark-to-production gap is not produced by any single deficiency but by the coupling among these five: fidelity gaps make surrogate metrics attractive, surrogate metrics make inexpensive improvement loops appear effective, and loops that optimise against surrogates accumulate behaviours that only surface as unsafe after deployment. The argument proceeds family by family, quantifying each where the literature permits, and then examines the trade-offs that arise when one family is addressed at the expense of another. It closes by identifying three distinct mechanisms through which these limitations convert into production failures, and by noting the conditions under which the cascade is dampened rather than amplified.

The cost family is the most frequently quantified and the least frequently remedied. CRAKEN augments a cybersecurity agent with knowledge-based execution and retrieval, yet the additional requests raise

operating cost while lifting the NYU CTF solve rate by only three percentage points, to 22%. Feedback-driven fuzzing of deep learning frameworks reports roughly $0.39 per bug and approximately $40.89 in total for a multi-agent configuration [23], while vulnerability-detection agents reach comparable conclusions about budget acting as the binding constraint [45]. The underlying mechanism is structural rather than incidental: self-improvement is a meta-loop in which every iteration pays for generation, execution, and critique, so cost grows linearly in rounds while capability gains are typically logarithmic. Marginal cost per unit of capability therefore rises monotonically, and production deployments, which operate under fixed per-query budgets, truncate the loop at precisely the round where the curve flattens. Benchmark evaluations, by contrast, report the endpoint of an unconstrained loop, which is why reported gains and realised gains diverge so sharply.

Environment fidelity is the family most directly implicated in the deployment gap. A longitudinal enterprise helpdesk study reports deflection rising from 38.0% to 61.3% as agents evolve, but its own limitations section states that the simulator simplifies concurrent load, network latency, malicious input, and cascading failure. This is more than an acknowledged caveat: because the simulator is the channel through which reward arrives, an evolving agent is incentivised to model the simulator rather than the task, and the resulting policy encodes precisely those regularities the simulator omits. The estimation error is consequently not random but directional, biased toward optimism along the axes of realism that were simplified. Closed-loop systems are especially exposed to this bias, since self-generated feedback amplifies whatever the environment rewards and provides no independent signal that a dimension of realism is missing. Fidelity, in other words, does not

10 DISCUSSION

The five questions we pursued converge on two threads that run through the entire literature. The first is architecture–task fit: self-improvement is not a single capability that a system either has or lacks, but a coupling between where a feedback signal originates, which artifact the system is permitted to revise, and when that revision is committed. Gains appear only when the revision locus matches the temporal structure of the task and the precision of the available signal. The second is measurement fragmentation: the field's claims about improvement, generalization, and transfer rest on heterogeneous and largely proxy-based instrumentation, so the boundaries we observe are partly boundaries of measurement rather than of method. These threads are not independent. Because the update target determines which signals can be exploited at all, and because the oracle determines which signals count as correct, any statement about what a mechanism can do is simultaneously a statement about what its instrumentation can see.

The first thread is most visible when the classification of mechanisms is read against the evidence on signal effectiveness. Mechanisms in the reviewed corpus occupy only a sparse subset of the space spanned by feedback source, update target, and commitment timing; no reviewed mechanism covers the full range of any one dimension, and the widest coverage comes from systems that revise model parameters, a reusable skill library, and persistent memory together, followed by systems that combine a composite-tool skill library, memory, and a trained policy. In contrast, several widely cited agent designs confine self-correction to prompt and context within a single episode, producing no artifact that survives the task [11]. This distribution reframes the question of which feedback signal is most effective. Signal quality is not an intrinsic property of a signal type; it is a property of the pairing between signal and update target. Revisions amortized into a durable artifact require high-precision, low-variance signals, because each error is written into something reused later. The same noisy signal that is tolerable inside an episode, where a wrong correction is discarded at termination, becomes corrupting once accumulated into a skill library or policy. The observed dependence of vulnerability-detection feedback on detector precision is therefore a special case of a more general condition on durable self-improvement, not an idiosyncrasy of security analysis.

This reading also disciplines a comparison that is easy to misread. Reported detection scores place retrieval-augmented detection above parameter-level adaptation, above a dual-agent configuration, and above the unassisted baseline [11]. But a verification-and-correction loop that never persists anything is not an improvement mechanism in the sense the other systems are; it is inference-time scaffolding. The ranking measures detection capability, not improvement capability, and should not be transferred across categories. The same caution applies to the recurrence of self-critique as a substitute for external verification: self-audit of generated code remains well below the reliability a closed loop demands, and repair success against a model's own output saturates well before the loop closes, which means self-generated feedback is a weak oracle precisely where no external check exists.

The second thread becomes sharpest where feedback and evaluation share an instrument. Static analyzers appear in the corpus in two roles at once: as the source of the signal that drives repair, and as the

judge that decides whether a vulnerability was removed. Both roles fail for the same reason. Against a thousand-plus manually labeled reference set, per-sample label agreement with human judgment falls to roughly two thirds, even though aggregate secure-classification rates look comparable or even favorable [50]; and in a controlled study of known weakness locations, scanner families identified essentially none of the planted weaknesses, while a language-model analysis recovered a substantial fraction [48]. Because aggregation conceals per-sample error, a detector can appear adequate as an oracle while injecting noise as a signal, and the two failures compound rather than cancel. The consequence cuts both ways for the literature's positive results. Closed-loop gains may be inflated where the loop exploits the same blind spots it is scored against, and genuine gains may be invisible where the reported metric is insensitive to the property of interest: compile rate varies substantially under a single compiler flag, can rank methods in the opposite order to a similarity-based metric, and awards non-trivial scores to degenerate patches that remove functionality [52]. Until the oracle question is settled, cross-paper comparisons of "improvement" are closer to incommensurable than to contested.

The distribution of benchmarks and the absence of security-specific oracles are likewise mutually reinforcing rather than independent deficiencies. Generation-side evaluation is locked onto function-level, single-language prompts with synthetic weakness specifications, with one benchmark family serving as the de facto hub across otherwise unrelated studies [53, 49, 6]; at least one of those studies states explicitly that its conclusions are scoped to a single language and may not extrapolate [49]. Benchmarks that could support executable validation—repository-scale, multi-language, drawn from real defects—are used only by a narrow repair lineage, and even those are drawn from well-maintained open-source projects, which likely flatters performance on legacy systems for which no dedicated benchmark exists [2]. Function-level synthetic prompts admit only pass/fail or classifier judgments, not exploit reproduction; exploit-grade validation requires the repository-scale artifacts that the generation literature almost never touches. The generalization boundary documented in our third question is therefore jointly methodological and instrumental: it is partly what the methods can do, and partly what the benchmarks were built to see. A related gap follows silently. Functional preservation is checked on a different population from the one on which security is assessed, so the field routinely reports security gains and functional correctness side by side without ever establishing that both hold for the same programs.

The question of transferability connects these threads by exposing what the other questions leave implicit. Across roughly a hundred assumption statements in the reviewed corpus, explicit dependencies between assumptions number fewer than ten, and most statements stand isolated; even constraints that papers state plainly in prose—perfect recall, a fixed inference budget, an identical base model across compared systems, single-language evaluation, an adversary capable of building its own agents—are not linked to the methods that depend on them [54]. This is systematic rather than incidental: methods are densely connected to the problems they address, the benchmarks they use, and the metrics they report, while the connection to their own preconditions is left unmodelled. The practical effect is that transfer failures are misattributed. When the assumptions that quietly travel with a result—a shared base model, a single language, a static adversary—are violated, the observed degradation is read as a limitation of the method rather than as an unlicensed extrapolation, which is exactly the confound that makes the generalization boundary of our third question so hard to interpret. Adversarial co-evolution is the hardest case: budget and adversary capability shift together in deployment, yet mechanisms are tuned with both held fixed, and no reviewed work represents the relaxation of either as an explicit dependency. Even the validity of a feedback signal is contingent here, since what counts as evidence of success depends on the budget under which the attacker operates.

Read together, the corpus supports four consensus findings and leaves two disagreements open. The consensus is that durable, cross-task self-improvement is rarer than nominal claims suggest; that the value of a feedback signal is bounded by its precision and by the durability of the artifact it updates; that security-specific, execution-grounded oracles are structurally absent rather than merely uncommon; and that assumptions are under-specified and almost never relaxed. The open disagreements concern whether model self-critique can substitute for external verification, and whether static analyzers are usable instruments at all—widely deployed as both signal and judge, yet nearly blind to known weakness locations in one controlled setting [48], while appearing broadly comparable at the aggregate level elsewhere [50]. We do not adjudicate this here, because the two findings differ in unit of analysis, and that difference is itself the methodological lesson: aggregate agreement and per-sample agreement answer different questions, and only the latter constrains whether a loop can be trusted.

The reusable contribution of this review is therefore threefold. First, a classification of self-improvement mechanisms along three empirically grounded dimensions—feedback source, update target, and commitment timing—together with the observed value sets on each and the sparsity of the resulting combinations;

sparsity is not a bookkeeping detail but a diagnostic, since a claim of self-improvement that occupies a single cell invites the question of which other cells its evidence would have to cover. Second, a coordinate system for assumptions built from access, budget, adversary capability, and environment fidelity, which turns transferability from a narrative caveat into an inspectable claim and makes relaxation an explicit modelling act rather than an unstated one. Third, a validity framework that separates execution-grounded security oracles from proxies and treats signal precision as a precondition on loop stability, replacing the question "does this method improve?" with the answerable pair "under what signal regime, and against what oracle?" Underlying all three is a single explanatory lens: architecture–task fit. It explains why the corpus's positive results cluster where durable artifacts meet precise signals, why its generalization claims outrun its benchmarks, and why its transfer failures are so often misread—and it converts a fragmented set of findings into design guidance: choose the update target from the temporal structure of the task and the precision of the feedback available, and report the oracle and the assumption set alongside the gain.

11 FUTURE RESEARCH

Research on agentic vulnerability discovery and repair has matured within distinct mechanism families, but the evidence base remains partitioned. Retrieval-oriented repair agents are typically evaluated on PatchEval, SEC-bench, or Vul4J, whereas autonomous penetration-testing scaffolds report on AutoPenBench, NYU-CTF, or CVE-Bench; defensive memory agents often rely on yet another set of harm-oriented benchmarks. This fragmentation is not merely a reporting inconvenience: it prevents cross-family claims about which architectural commitments—retrieval, planning, or persistent memory—actually drive gains. A balanced reading treats benchmark heterogeneity as an opportunity to construct shared, stratified evaluation suites that deliberately cross mechanism families and task difficulty. Recent work on repair metrics and on test suites as improvement signals offers candidate instruments for such suites, but neither has been calibrated across retrieval, scaffolding, and defense-memory families. Without that calibration, comparative claims remain local and difficult to accumulate.

A second gap concerns the evidential status of execution feedback. Compiler diagnostics, sanitizers, and test suites have become the de facto signal for iterative patch refinement, and this design has been inherited by later repair agents such as EviRepair and repository-level memory systems. Yet the field has not established when these signals are sufficient. deepSURF shows that fuzzing plus sanitizers can find memory-safety bugs in Rust crates, but its static harness-selection strategy leaves coverage-guided exploration underexploited; conversely, execution feedback without coverage prioritization may repeatedly confirm the same localized behavior. The critical point is that current evidence supports the usefulness of execution signals, not a general theory of which signals supervise which repair decisions. A stronger agenda would combine dynamic analysis with coverage objectives and test complementarity explicitly, including recent efforts to combine LLM reasoning with dynamic analysis for security testing. Until such ablations exist, claims that execution feedback broadly improves repair quality should be treated as provisional.

Evaluation assumptions constitute a third

12 LIMITATIONS

The corpus combines an automated retrieval pool with manually added sources. This design broadens coverage but introduces coverage bias: the automated pool reflects the indexing and ranking behavior of the queried databases, while manual supplementation reflects reviewer judgment about relevance and salience. Sources outside indexed venues, non-English publications, very recent work, and grey literature may therefore be underrepresented. The review applies a fixed time cutoff; work published or indexed after that cutoff is not included, and later developments may alter the evidentiary picture.

The review uses model-assisted extraction and drafting. To support verifiability, each claim is annotated with the literature that supports it, and the analysis retains a trace of extraction and synthesis decisions. These measures support auditability but do not eliminate error: individual extractions may misread, omit, or overgeneralize details from source studies, and model assistance can propagate such errors into summaries. The annotations identify support for claims; they do not constitute independent re-analysis of primary data.

The review is scoped to the stated topic and treats adjacent fields only where they intersect with that topic. It does not survey neighboring literatures in their own right, so findings and methods developed primarily in those fields appear only when they bear directly on the review question. This scope sharpens the synthesis but excludes adjacent work that may inform related questions outside the stated boundary. The five lines of inquiry converge on an answer narrower than the question that opened this paper. Self-improvement for safety tasks is not a single capability a model either possesses or lacks; it decomposes,

and the decomposition is empirical rather than stipulated. Feedback-signal source, iteration structure and temporal scope vary independently across the literature, and the citation graph preserves that independence — 277 ‘extends‘ edges hang from just four mechanism prototypes, a hub-and-spoke lineage with a very narrow base. Four prototypes carrying several hundred extensions is itself a result: applications have multiplied considerably faster than mechanisms.

The temporal axis is the lopsided one. Single-round feedback and multi-round iteration are evidence-saturated, and cross-task experience accumulation is supported solidly enough to be treated as settled rather than provisional. Cross-version persistent evolution is neither: the evidence is sparse, confined almost entirely to simulation or to short time windows, and the obstacle is mechanical rather than conceptual, since iteration budgets are hard-coded instead of adaptive. By construction, nothing in the current designs accumulates across versions.

Effectiveness tracks signal type far more closely than architecture. Vulnerability-detection feedback buys improvement strictly in proportion to detector precision, and in the absence of an explicit safety specification to anchor the loop, alarm noise rather than model weakness becomes the dominant destabilizer; the gains here are real but borrowed from the detector, not produced by the loop. Program-execution feedback is the one signal repeatedly validated as a direct driver of measurable improvement, and it should be weighted accordingly, above coverage signals, which have not earned comparable confidence. Offensive–defensive experience signals stand apart: different evaluation paradigms, metric semantics and environment assumptions make them not strictly comparable with vulnerability-side and repair-side results, and pooling the two into a single effectiveness claim would be a mistake the evidence cannot underwrite.

What these mechanisms can support is bounded by the same evidence that supports them. Distributional skew is confirmed systematically rather than incidentally, and it caps how far the demonstrated reach of any one mechanism can be extended beyond the settings in which it was demonstrated.

REFERENCES

[1] Avinash Anand, Akshit Gupta, Nishchay Yadav, and Shaurya Bajaj. A comprehensive survey of ai-driven advancements and techniques in automated program repair and code generation, 2026. Preprint.

[2] Harsh Verma. Automated vulnerability patching in legacy code using llms and multi-ai agents, 2026. Preprint.

[3] Yutao Mou, Xiao Deng, Yuxiao Luo, Shikun Zhang, and Wei Ye. Can you really trust code copilots? evaluating large language models from a code security perspective. Preprint.

[4] Jiexin Wang, Liuwen Cao, Xitong Luo, Zhiping Zhou, Jiayuan Xie, Adam Jatowt, and Yi Cai. Enhancing large language models for secure code generation: A dataset-driven study on vulnerability mitigation, 2026. Preprint.

[5] Vidyut Sriram, Sawan Pandita, Achintya Lakshmanan, Aneesh Shamraj, and Suman Saha. Improving llm-assisted secure code generation through retrieval-augmented-generation and multi-tool feedback, 2026. Preprint.

[6] Ana Nunez, Nafis Tanveer Islam, Sumit Kumar Jha, and Peyman Najafirad. Autosafecoder: A multi-agent framework for securing llm code generation through static analysis and fuzz testing, 2026. Preprint.

[7] Jugal Gajjar, Kamalasankari Subramaniakuppusamy, Relsy Puthal, and Kaustik Ranaware. Securefixagent: A hybrid llm agent for automated python static vulnerability repair, 2026. Preprint.

[8] A case study of llm for automated vulnerability repair, 2026. Preprint.

[9] Association for Computational Linguistics 2026, Yu Huang, Kevin Leach, J Li, Kexin Pei, and Y Zhang. Synthfix: Adaptive neuro-symbolic code vulnerability repair. Preprint.

[10] Chengyan Ma, Jieke Shi, Ruidong Han, Ye Liu, Feng Li, Yuqing Niu, and David Lo. Automated repair of tee partitioning issues via dsl-guided and llm-assisted patching. Preprint.

[11] Md Hasan Saju, Maher Muhtadi, and Akramul Azim. An empirical evaluation of llm-based approaches for code vulnerability detection: Rag, sft, and dual-agent systems, 2026. Preprint.

[12] Reza Fayyazi, Stella Hoyos Trueba, Michael Zuzak, and Shanchieh Jay Yang. Proverag: Provenance-driven vulnerability analysis with automated retrieval-augmented llms, 2026. Preprint.

[13] Tenghui Huang, Jinbo Wen, Jiawen Kang, Siyong Chen, Zhengtao Li, Tao Zhang, Dongning Liu, Jiacheng Wang, Chengjun Cai, Yinqiu Liu, and Dusit Niyato. Paravul: A parallel large language model and retrieval-augmented framework for smart contract vulnerability detection. Preprint.

[14] Yanli Jin, Chunpei Li, Peng Fan, Peng Liu, Xianxian Li, Chen Liu, and Wangjie Qiu. Llm-bscvm: An llm-based blockchain smart contract vulnerability management framework. Preprint.

[15] Xiangmin Shen, Lingzhi Wang, Zhenyuan Li, Yan Chen, Wencheng Zhao, Dawei Sun, Jiashui Wang, and Wei Ruan. Pentestagent: Incorporating llm agents to automated penetration testing, 2026. Preprint.

[16] Ibrahim Alshehri, Adnan Alshehri, Abdulrahman Almalki, Majed Bamardouf, and Alaqsa Akbar. Breachseek: A multi-agent automated penetration tester. Preprint.

[17] Phung Duc Luong, Le Tran Gia Bao, Nguyen Vu Khai Tam, Dong Huu Nguyen Khoa, Nguyen Huu Quyen, Van-Hau Pham, and Phan The Duy. xoffense: An autonomous multi-agent framework for penetration testing with domain-adapted large language models, 2026. Preprint.

[18] Refpentester: A knowledge-informed self-reflective penetration testing framework, 2026. Preprint.

[19] Minghao Shao, Haoran Xi, Nanda Rani, Meet Udeshi, Venkata Sai Charan Putrevu, Kimberly Milner, Brendan Dolan-Gavitt, Sandeep Kumar Shukla, Prashanth Krishnamurthy, Farshad Khorrami, Ramesh Karri, and Muhammad Shafique. Craken: Cybersecurity llm agent with knowledge-based execution, 2026. Preprint.

[20] Yizhou Yang and Xin Liu. Behaviour-diverse automatic penetration testing: A curiosity-driven multi-objective deep reinforcement learning approach. Preprint.

[21] Nafis Tanveer Islam, Joseph Khoury, Andrew Seong, Mohammad Bahrami Karkevandi, Gonzalo De La Torre Parra, Elias Bou-Harb, and Peyman Najafirad. Llm-powered code vulnerability repair with reinforcement learning and semantic reward. Preprint.

[22] Chaofan Shou, Jing Liu, Doudou Lu, and Koushik Sen. Llm4fuzz: Guided fuzzing of smart contracts with large language models, 2026. Preprint.

[23] May the feedback be with you! breaking the seal of feedback-driven deep learning framework fuzzing via large language models, 2026. Preprint.

[24] Georgios C. Androutsopoulos and Antonio Bianchi. deepsurf: Detecting memory safety vulnerabilities in rust through fuzzing llm-augmented harnesses, 2026. Preprint.

[25] Cohen and Avihay. In-browser llm-guided fuzzing for real-time prompt injection testing in agentic ai browsers. Preprint.

[26] Yingkai Dong, Xiangtao Meng, Ning Yu, Zheng Li, and Shanqing Guo. Fuzz-testing meets llm-based agents: An automated and efficient framework for jailbreaking text-to-image generation models, 2026. Preprint.

[27] Weidi Luo, Shenghong Dai, Xiaogeng Liu, Suman Banerjee, Huan Sun, Muhao Chen, and Chaowei Xiao. Agrail: A lifelong agent guardrail with effective and adaptive safety detection, 2026. Preprint.

[28] Zombie agents: Persistent control of self-evolving llm agents, 2026. Preprint.

[29] Ninghui Li, Kaiyuan Zhang, Kyle Polley, and Jerry Ma. Security considerations for artificial intelligence agents, 2026. Preprint.

[30] Dasen Fu and Dalin Xiang. The security duality of large language model agents: A systematic review of self-security protection and cybersecurity empowerment, 2026. Preprint.

[31] Kunal Pai, Parth Shah, and Harshil Patel. Naamse: Framework for evolutionary security evaluation of agents, 2026. Preprint.

[32] Agentic context engineering: Evolving contexts for self-improving llm systems, 2026. Preprint.

[33] Lin Fan, Pengyu Dai, Zhipeng Deng, Haolin Wang, Xun Gong, Yefeng Zheng, and Yafei Ou. Evolving medical imaging agents via experience-driven self-skill discovery, 2026. Preprint.

[34] Wei Huang, Hai-Feng Ling, Linyuan Bai, and De-Tian Chu. Evomail: Self-evolving cognitive agents for adaptive spam and phishing email defense, 2026. Preprint.

[35] Jiahao Chen, Zhou Feng, Oubo Ma, Yichen Yan, Ruixiao Lin, Hangtao Zhang, Linkang Du, Hengyu An, Yong Yang, Jun Liu, Junhao Li, and Naen Xu. Safety in self-evolving agents: A survey, 2026. Preprint.

[36] Dipayan Saha, Hasan Al Shaikh, Shams Tarek, and Farimah Farahmandi. Threatlens: Llm-guided threat modeling and test plan generation for hardware security verification. Preprint.

[37] Smartauditflow: A dynamic plan-execute framework for smart contract security auditing, 2026. Preprint.

[38] Xiaoyin Chen, Canwen Xu, Yite Wang, Boyi Liu, Zhewei Yao, and Yuxiong He. Learning to self-evolve. Preprint.

[39] He Kong, Die Hu, Jingguo Ge, Liangxiong Li, Tong Li, and Bingzhen Wu. Vulnbot: Autonomous penetration testing for a multi-agent collaborative framework, 2026. Preprint.

[40] Yuchong Xie, Mingyu Luo, Zesen Liu, Zhixiang Zhang, Kaikai Zhang, Yu Liu, Zongjie Li, Ping Chen, Shuai Wang, and Dongdong She. Red-teaming coding agents from a tool-invocation perspective: An empirical security assessment, 2026. Preprint.

[41] Hao Yan, Swapneel Suhas Vaidya, Xiaokuan Zhang, and Ziyu Yao. Guiding ai to fix its own flaws: An empirical study on llm-driven secure code generation, 2026. Preprint.

[42] Mingming Zhang, Xu Wang, Jian Zhang, Xiangxin Meng, Jiayi Zhang, and Chunming Hu. Vulnresolver: A hybrid agent framework for llm-based automated vulnerability issue resolution. Preprint.

[43] Yihe Fan, Changyi Li, Lichen Xu, Xudong Pan, Jiarun Dai, Hong Geng, and Min Yang. Cyberevolver: Structured self-evolution for cybersecurity agents on the fly, 2026. Preprint.

[44] Simiao Liu, Li Zhang, Fang Liu, Xiaoli Lian, Yang Liu, and Yinghao Zhu. Memrepair: Hierarchical memory for agentic repository-level vulnerability repair, 2026. Preprint.

[45] Yarra and Rajesh. Llmpatronous: Harnessing the power of llms for vulnerability detection, 2026. Preprint.

[46] R Uma, B Shravani, M Shivani, C Komal, and N Dinesh Kumar. Self-evolving ai agent with skill memory, 2026. Preprint.

[47] Ziyi Yang, Kalit Inani, Keshav Kabra, Vima Gupta, and Anand Padmanabha Iyer. Safuzz: Semantic-guided adaptive fuzzing for llm-generated code, 2026. Preprint.

[48] Md. Wasiul Haque, Sagar Dasgupta, and Mizanur Rahman. Llmsec-av: A vulnerability taxonomy and llm-driven software weakness discovery framework for autonomous vehicles, 2026. Preprint.

[49] Jianian Gong, Nachuan Duan, Ziheng Tao, Zhaohui Gong, Yuan Yuan, and Minlie Huang. How well do large language models serve as end-to-end secure code agents for python?, 2026. Preprint.

[50] Ehsan Firouzi and Mohammad Ghafari. Persistent human feedback, llms, and static analyzers for secure code generation and vulnerability detection, 2026. Preprint.

[51] Piotr Przymus, Andreas Happe, and Jürgen Cito. Adversarial bug reports as a security risk in language model-based automated program repair, 2026. Preprint.

[52] Om Nepal, Sushant Aryal, Oluseyi Olukola, and Nick Rahimi. Metrics failure in llm-based code vulnerability repair: An empirical study and a change-aware screen, 2026. Preprint.

[53] Marc Bruni, Fabio Gabrielli, Mohammad Ghafari, and Martin Kropp. Benchmarking prompt engineering techniques for secure code generation with gpt models, 2026. Preprint.

[54] Yuxuan Liu, Zhaochen Su, Yuhao Zhang, Jiahe Guo, Zhongwei Xie, Huihao Jing, Lingyun Xie, Qing Zong, Yauwai Yim, Zhixiong Zhang, Haoran Li, and Yangqiu Song. Rethinking self-evolving agent skills: Feedback dynamics over multiple rounds, 2026. Preprint.